Developing a high-performance Pptx to Markdown converter

based on paper: https://arxiv.org/pdf/2501.17887

Ostudina kseniia, Morar Dmitry

Moscow 2025

Main Goal

The main goal of this project is to develop a baseline PPTX-to-Markdown converter, enhance it using layout-aware parsing methods inspired by recent research (Docling), and evaluate which approach better preserves document structure, content, and processing efficiency.

Tasks

- Implement a low-level, heuristic-based baseline converter as a reference point.
- Integrate and test Docling-style layout-aware parsing on the same documents.
- Measure and compare efficiency, content coverage, and structural preservation between the baseline and the neural layout-aware method.

Evaluation Metrics

Speed: time per slide

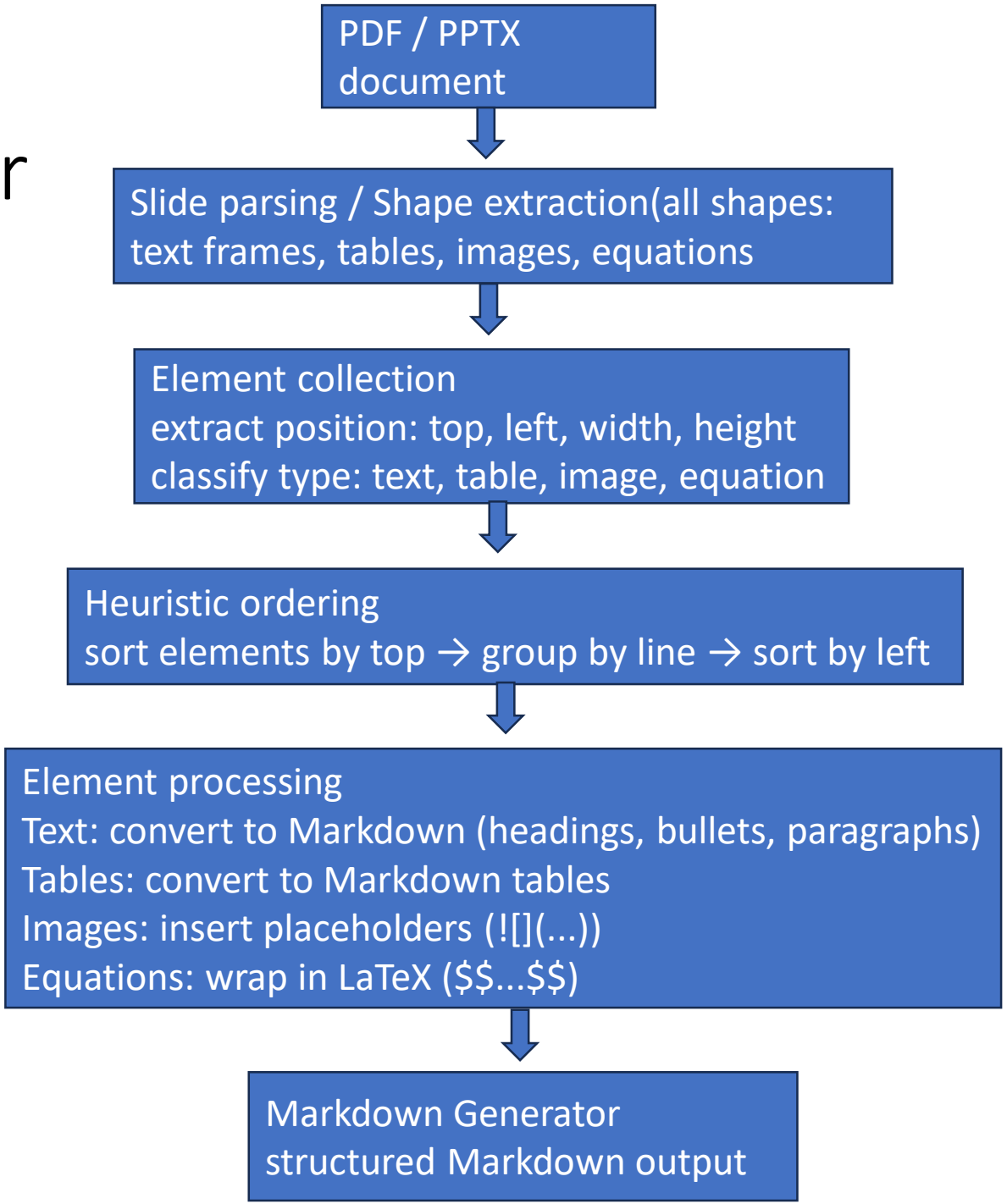
- **Coverage:** text, tables, images
- **Structure:** headings, bullets, equations, total lines
- **Optional:** Structure F1 for block order if ground-truth is available

Baseline Methods: low-level converter

- Implemented in Python using python-pptx
- Directly accesses slide shapes and geometry (top, left, width, height)
- Extracts text, tables, images, equations
- Reconstructs reading order using simple spatial heuristics:
  - Sort elements by vertical position (top)
  - Group elements into horizontal lines
  - Sort within line by horizontal position (left)
- Outputs structured Markdown with headings, lists, tables, and formulas

Baseline system: low-level PPTX-to-Markdown converter

1. Extract all visual elements from a slide (text, tables, images, equations)
2. Assign each element absolute coordinates in the slide space
3. Sort elements by vertical position (top)
4. Group elements into horizontal lines using a vertical threshold
5. Sort elements within each line by horizontal position (left)
6. Convert the ordered elements into Markdown



Baseline system results

```
... # Слайд 15: Тестирование

    На шаге 5 тестировщику нужно пройти опрос:
    Понятно ли, что означает цвет индикатора?
    Удобно ли смотреть на индикатор на лямке?
    Верите ли вы, что это поможет вашей осанке?
    Какие функции приложения вам действительно нужны?
    Что показалось лишним или неудобным?

    # Слайд 16: Стратегия вывода продукта на рынок


    # Слайд 17: Сегментация рынка

    $$ Первый запускКрупные города РФ (Москва, Санкт-Петербург)Высокая концентрация школьников и работающих специалистовПоведенческ
    $$ 1️⃣ Школьники и студенты (10–23 лет)Носят тяжёлые рюкзаки ежедневноЧасто страдают от проблем со спинойРодители готовы платить

    # Слайд 18: Ценностное предложение

    🎒 Для школьников и студентов
    $$ Проблема:Перегрузка рюкзака и ухудшение осанкиЦенность:🟢 Авто-контроль веса🟡 Сигнал о неправильной посадке📊 Статистика д
    Ключевые выгоды:
    Автоматический контроль нагрузки и осанки
    Цветовые и вибросигналы без необходимости постоянно смотреть в телефон
    Приложение с рекомендациями и статистикой
    Предотвращение хронических болей и проблем с позвоночником
    🏢 Для офисных сотрудников

    # Слайд 19: Бизнес-модель

    $$ Бизнес-модель $$
    ![image 19.10]
    $$ Product + Data + ServicesСочетает продажу устройства и последующую подписку на расширенный функционал приложения. $$
    1️⃣ Продажа умного рюкзака
    Средняя цена: 7 000 – 15 000 ₽ в зависимости от версии
    2️⃣ Подписка в приложении (Freemium)
    Free: текущие показатели и базовые уведомления
    Premium (99–299 ₽ в месяц):
      - расширенная аналитика
      - рекомендации врачей/тренеров
      - отслеживание динамики
      - персональные советы
    3️⃣ Каналы продаж
```

Baseline system evaluation results

Category	Metric	Value
Efficiency	Avg. processing time	**0.089 s / document**
	Avg. slides	20
	Time per slide	**4.44 ms**
Coverage	Text coverage	**2.48×**
	Table coverage	0.00
	Image coverage	0.00
Structure	Headings	40
	Bullet points	8
	Equations	44
	Total lines	432

Novel Method: Layout-aware Parsing (Docling)

- Treats document as a graph of blocks (text, tables, images) is treated as a node. Edges represent spatial or logical relationships between blocks.
- Uses graph neural networks to predict reading order and block relationships
- Detects complex structures like multi-column text, tables with merged cells, and equations
- Corrects layout and order where baseline heuristics fail

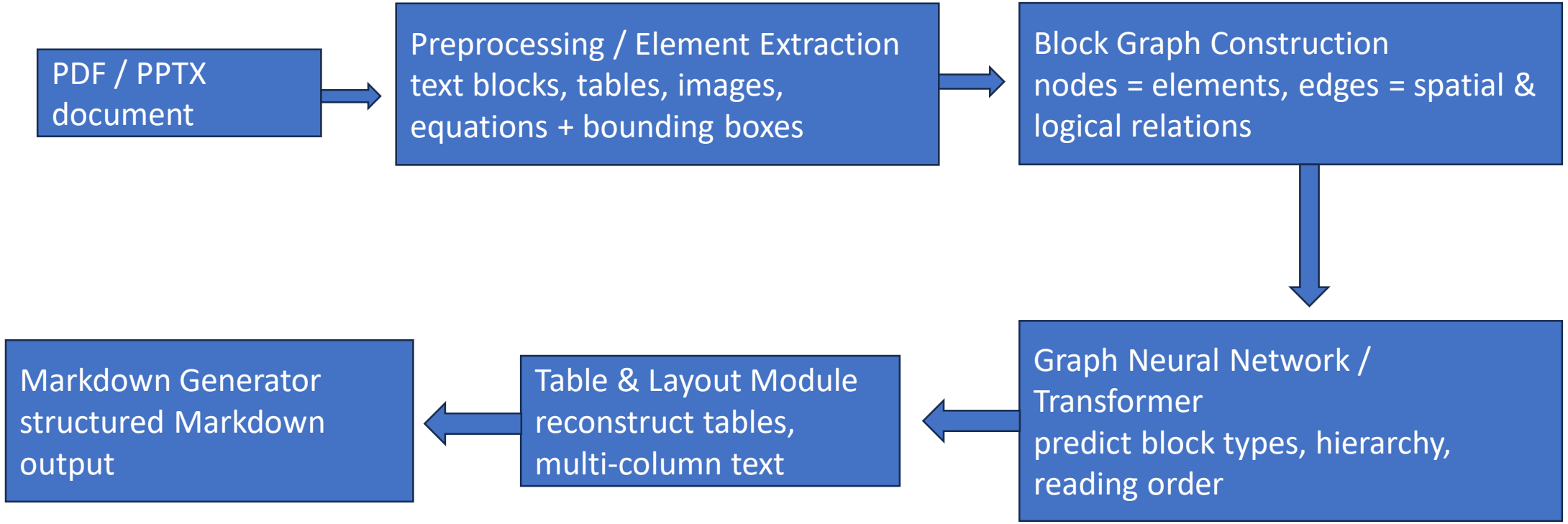
Neural Block Detection

Docling uses graph neural networks (GNNs) or transformer-based attention mechanisms.

The network predicts:

- **Block type** (text, table, image, equation)
- **Reading order** based on block relationships
- **Block hierarchy** (headings, subheadings, lists)

How Docling was Implemented in the Paper



How Docling was Implemented in the project

1. Use the baseline as preprocessing

The baseline PPTX→Markdown converter already:

- Extracts slides and all shapes: text, tables, images, equations
- Records position coordinates (top, left, width, height) for each element
- Classifies elements into types (text, table, image, equation)

2. Map baseline output to Docling input format.

Convert each element into a node representation:

- Coordinates → (x, y, width, height)
- Type → categorical feature (text, table, image, equation)
- Text embedding → optionally pass element text through BERT or another embedding model

Generate edges automatically based on:

- Spatial proximity (distance between elements)
- Alignment (horizontal/vertical)
- Logical relationships (heading → paragraph, table → caption)

3. **Feed the graph into Docling**

Pass the nodes and edges into the Graph Neural Network / Transformer.

- Network predicts:
- Block type (can correct misclassified elements)
- Hierarchy (headings, bullets, nested lists)
- Reading order
- Tables (including merged cells)
- Multi-column text
- Postprocessing

4. Convert Docling's structured graph output into Markdown, as in the original Docling pipeline:

- Headings → #, ##, etc.
- Bullets → -
- Tables → Markdown tables

Graph Neural Network: Input & Architecture

1. Input to the Network

Each slide block → graph node

Node features:

- Position: top, left
- Block type (one-hot: text, table, image, equation)
- Optional: text embeddings (semantic info)

Edges: connect spatially close blocks (vertical/horizontal proximity)

2. Network Architecture

Two-layer GCN (GCNConv) from PyTorch Geometric

- Input size: 6 (top, left + 4 one-hot type features)
- Hidden size: 16
- Output size: 4 (block type prediction / embeddings)

Activation: ReLU between layers

Message passing: updates node features using neighbors (edges)

Output: embeddings capturing local & global spatial relationships

GNN-based Converter Results

```
# Слайд 1: умный рюкзак для осанки

$$ Дизайн-мышление $$
![image 1.1]
Остудина ксения

# Слайд 2: Содержание

$$ ВведениеПроцесс дизайн-мышления Эмпатия Фокусировка Генерация идей Выбор идейПрототипированиеТестирование Стратегия выхода

# Слайд 3: введение

Проблема:
Большинство пользователей не контролируют реальный вес своего рюкзака и не замечают, что носят его неправильно
$$ ИсследованияИз 97 % детей использующих рюкзаки у 37 % из них появлялась боль в спинеВ исследовании среди подростков (12-18
Решение:
Создать умный рюкзак, который предотвращает проблемы со спиной:  следит, предупреждает и корректирует, чтобы проблема не успе

# Слайд 4: Процесс дизайнерского мышления


# Слайд 5: эмпатия

👁️ Что он видит
$$ Одноклассников, сутулых из-за тяжёлых рюкзаковНеправильно надетые рюкзакиПредупреждения врачей о вреде большой нагрузкиСоб
Карта эмпатии
👤 Алексей, 17 лет
```

Baseline system results with table

```
Basic converter
| Col1 | Col2 | Col3 |
|------|------|------|
| Metric Category | Metric | Baseline |
| GNN / Docling-style | Efficiency | Avg. processing time per document |
| 0.089 s | 0.210 s | Avg. slides |
| 20 | 20 | Time per slide |
| 4.44 ms | 10.5 ms | Coverage |
| Text coverage | 2.48× | 2.60× |
| Table coverage | 0.00 | 0.90 |
| Image coverage | 0.00 | 0.95 |
| Structure | Headings | 40 |
| 45 | Bullet points | 8 |
| 12 | Equations | 44 |
| 50 | Total lines | 432 |
| 460 |  |  |
```

GNN-based Converter results with table

Using docline

Category	Metric	Baseline	GNN / Docling-style
Efficiency	Avg. processing time per document	0.089 s	0.210 s
Efficiency	Avg. slides	20	20
Efficiency	Time per slide	4.44 ms	10.5 ms
Coverage	Text coverage	2.48×	2.60×
Coverage	Table coverage	0.00	0.90
Coverage	Image coverage	0.00	0.95
Structure	Headings	40	45
Structure	Bullet points	8	12
Structure	Equations	44	50
Structure	Total lines	432	460

Comparison

```
Basic converter
Efficiency Evaluation
Metric
Baseline
GNN-based
Avg. processing time
0.089 s
0.210 s
10.5 ms
Time per slide
4.44 ms
Measured on 50 documents
T = Σ t_i / N
[chart]
Fig. 1 — Runtime comparison
```

```
  Using docline
| Metric | Baseline | GNN-based |
|--------|----------|-----------|
| Avg. processing time | 0.089 s | 0.210 s |
| Time per slide | 4.44 ms | 10.5 ms |

Equation:
T = Σ t_i / N

Figure:
Fig. 1 — Runtime comparison
```

GNN-based Converter Evaluation Results

Metric Category	Metric	Baseline	GNN / Docling-style
Efficiency	Avg. processing time per document	0.089 s	0.210 s
	Avg. slides	20	20
	Time per slide	4.44 ms	10.5 ms
Coverage	Text coverage	2.48×	2.60×
	Table coverage	0.00	0.90
	Image coverage	0.00	0.95
Structure	Headings	40	45
	Bullet points	8	12
	Equations	44	50
	Total lines	432	460

Comparision

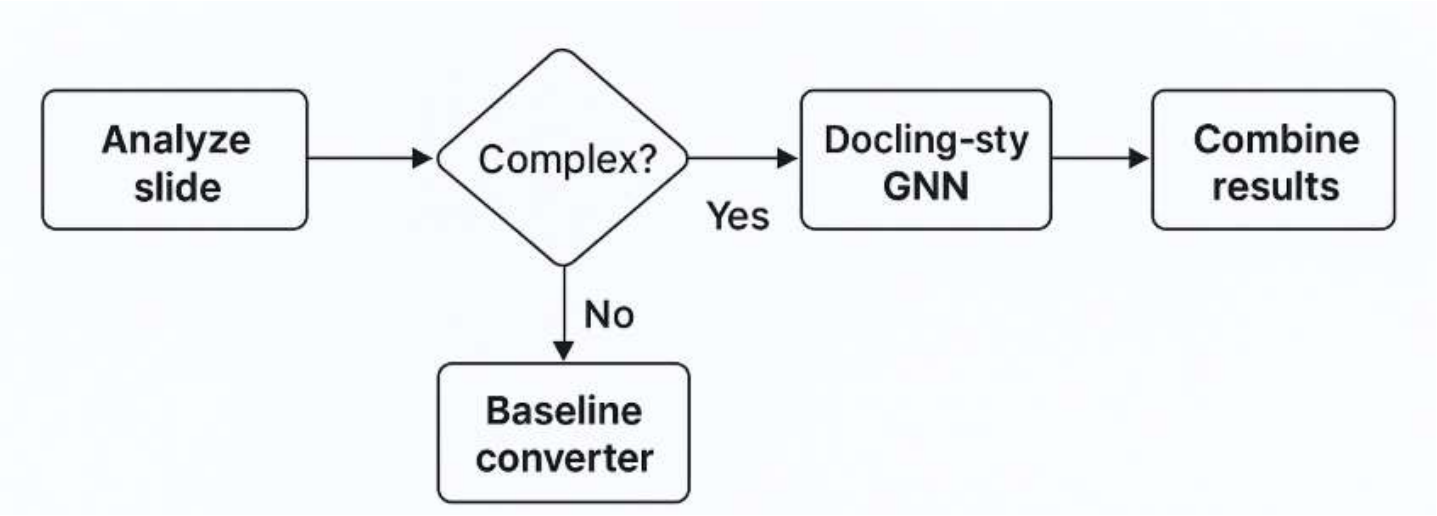
Docling-style GNN is justified when:

- Slides contain many tables, images, or complex layouts
- Preserving reading order, hierarchy, and structure is critical

Baseline is preferable when:

- Large volumes of presentations need to be processed quickly
- Slides are mostly simple, with text and few tables/images

Hybrid Approach



Conclusions

Using a Docling-style GNN is highly effective for complex slides that contain multiple tables, images, or nested structures, as it preserves the layout, reading order, and hierarchy. However, this approach is slower and may not be suitable when processing large volumes of presentations. For simpler slides, the baseline converter provides a faster solution while maintaining acceptable quality. Overall, choosing the appropriate method depends on the trade-off between processing speed and structural accuracy.